RecuVeicol

Gestione del soccorso stradale

Project work · ITS Developer 4.0
Novembre 2026



Team
Vincenzo Minervini · Francesco Valeriano
Giuseppe Cangellosi · Marika Sancilio · Marco Colucci

Cosa dire a voce

Buongiorno, vi presentiamo RecuVeicol, il nostro project work del corso ITS Developer 4.0. È un sistema che gestisce il soccorso stradale dal primo messaggio del cliente fino alla chiusura dell'intervento. Vedremo il problema, la soluzione, le tecnologie scelte e poi il sistema in funzione.

In parole semplici

RecuVeicol è il marchio; FORNITORI è il nome tecnico del repository.

File principali

`README.md`

APERTURA

Team e ruoli

Area	Responsabili
Backend	Vincenzo Minervini
Frontend	Francesco Valeriano, Giuseppe Cangellosi, Marika Sancilio
Chatbot	Marco Colucci
Mappe	Marco Colucci
Notifiche	Marika Sancilio, Francesco Valeriano, Giuseppe Cangellosi

Cosa dire a voce

Il lavoro è diviso in cinque aree. Vincenzo cura il backend e il database. Francesco, Giuseppe e Marika curano il frontend e le notifiche. Marco cura il chatbot e le mappe. Ogni area ha i propri file e comunica con le altre solo tramite API.

In parole semplici

Ogni area ha un responsabile; le parti si parlano solo tramite API.

File principali

`docs/team/README.md`

APERTURA

Il problema

- Chi resta in panne vuole chiedere aiuto subito, dal telefono
- La centrale deve trovare un autista libero e vicino
- Il cliente vuole sapere quando arriva l'aiuto

Cosa dire a voce

Chi resta fermo per strada vuole chiedere aiuto dal telefono e sapere che qualcuno sta arrivando. La centrale deve capire dove si trova il cliente e scegliere un autista libero e vicino. Se l'autista non risponde, la richiesta non deve fermarsi. Il problema è coordinare queste tre persone tenendo i dati coerenti.

In parole semplici

Cliente, centrale e autista devono coordinarsi su canali diversi.

File principali

`docs/requirements/BUR_FORNITORI.md`

APERTURA

La soluzione

- **Telegram**: il cliente apre la richiesta
- **Sito web**: la centrale controlla, l'autista interviene
- **Server unico**: regole e costi in un solo posto



Sito pubblico RecuVeicol

Cosa dire a voce

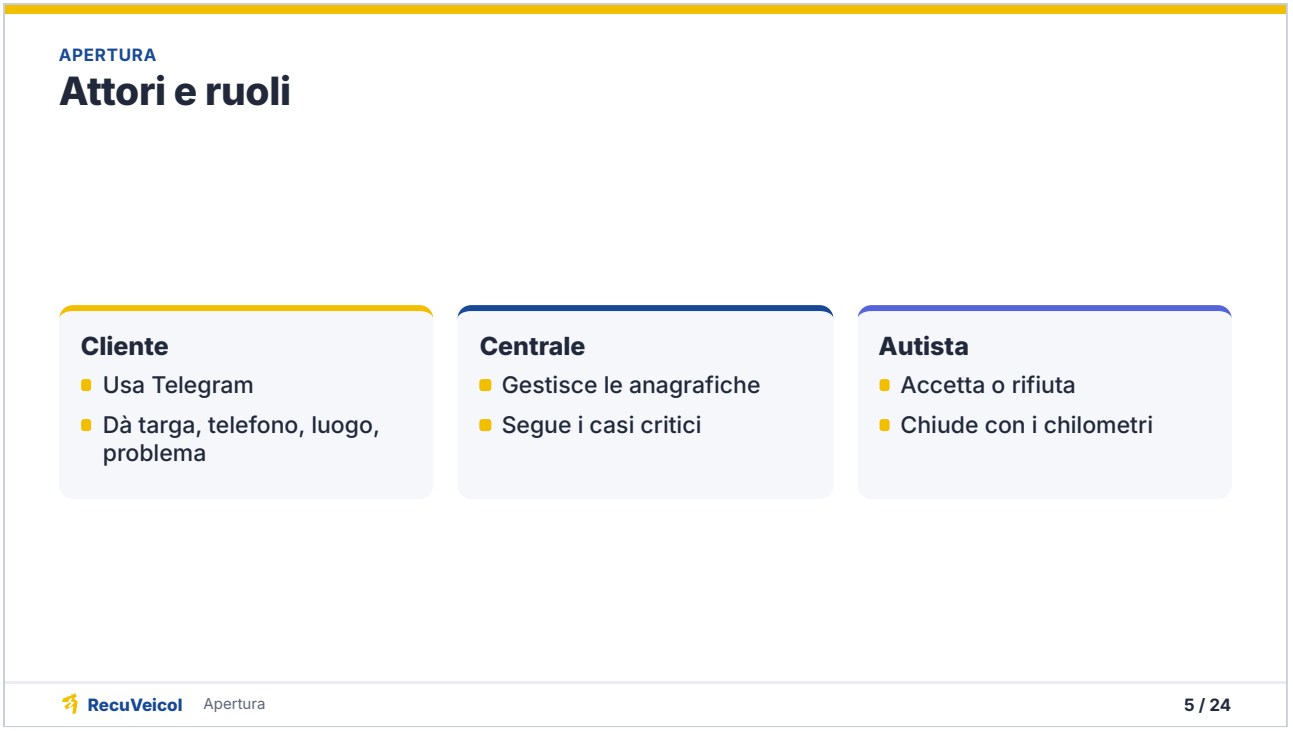
Il cliente scrive al bot su Telegram, senza installare altro. Il server la propone all'autista più vicino e la centrale segue i casi critici. L'autista lavora dalla stessa applicazione web, anche da telefono: accetta, vede la mappa e chiude l'intervento. Tutte le regole stanno sul server, e il cliente riceve gli aggiornamenti in chat.

In parole semplici

Il cliente usa Telegram, centrale e autista il sito; decide sempre il server.

File principali

`README.md` · `docs/architecture/TAD_FORNITORI.md`



Cosa dire a voce

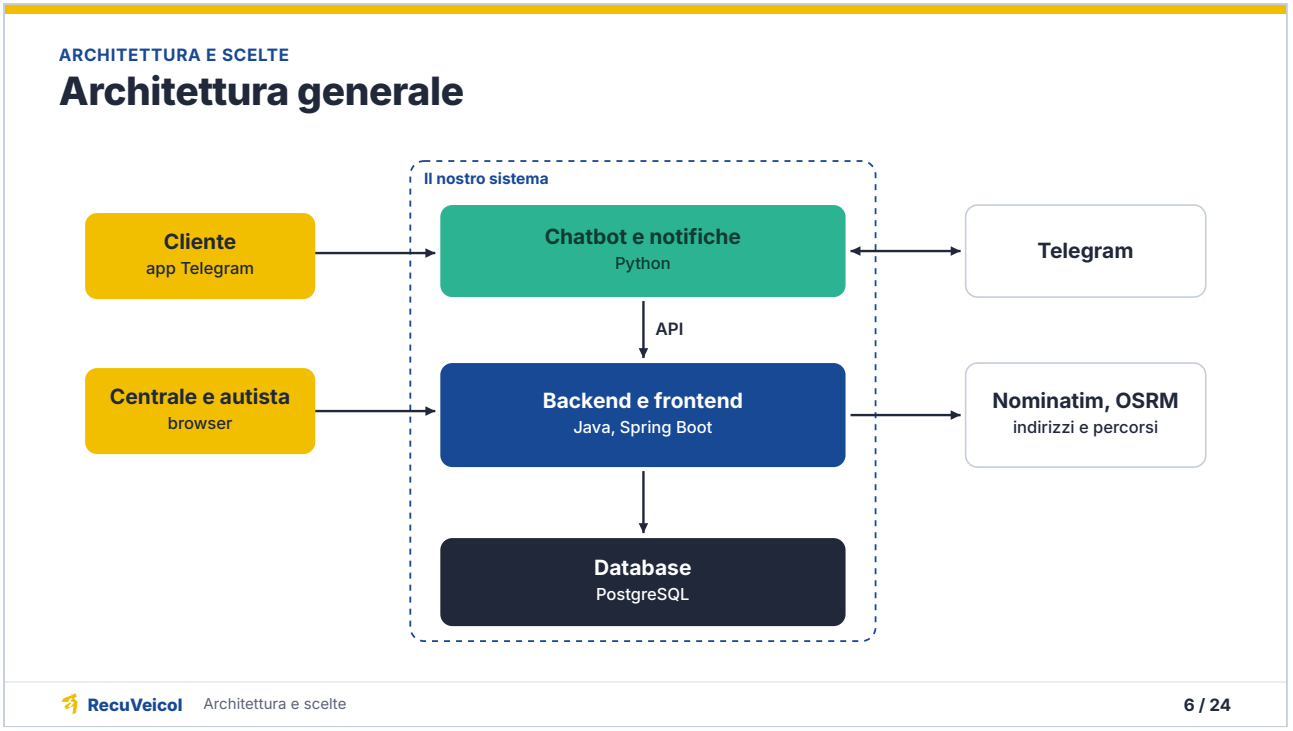
Gli attori sono tre. Il cliente non ha un account e scrive al bot. La centrale gestisce anagrafiche e missioni. L'autista vede solo le proprie missioni: accetta o rifiuta entro dieci minuti e alla fine chiude con i chilometri. I ruoli sono controllati dal server a ogni richiesta.

In parole semplici

Centrale e autista hanno un login con un ruolo; il cliente usa solo il bot.

File principali

`config/SecurityConfig.java` · `docs/manuals/MANUALE_UTENTE.md`



Cosa dire a voce

Al centro c'è il backend, l'unico componente che parla con il database. Serve anche le pagine web a centrale e autista. Il bot Python riceve i messaggi da Telegram e chiama il backend tramite API. Per indirizzi e percorsi il backend usa due servizi esterni, Nominatim e OSRM.

In parole semplici

Un solo server centrale: bot e browser non toccano mai il database.

File principali

`docs/architecture/TAD_FORNITORI.md`

ARCHITETTURA E SCELTE

Tecnologie: cosa e perché

Livello	Tecnologia	Linguaggio	Perché
Frontend	Bootstrap, Leaflet	JavaScript	Nessuna compilazione
Backend	Spring Boot	Java 25	Sicurezza e dati integrati
Database	PostgreSQL 18	SQL	Vincoli e transazioni
Chatbot	Telegram	Python	Nessuna dipendenza
Mappe	Nominatim, OSRM	Java, JavaScript	Open source e gratuiti
Notifiche	Outbox e dispatcher	Java, Python	Messaggi in coda sicura
Build	Maven, GitHub Actions	XML, YAML	Stessa build per tutti

Cosa dire a voce

Questa tabella riassume lo stack. Il frontend non richiede compilazione. Spring Boot offre già sicurezza e accesso ai dati. PostgreSQL protegge i dati con vincoli e transazioni. Il bot usa solo la libreria standard di Python. Mappe gratuite e senza chiavi. Maven garantisce a tutti la stessa build.

In parole semplici

Ogni tecnologia ha un motivo pratico legato al progetto.

File principali

`pom.xml` · `.github/workflows/backend-ci.yml` · `api/openapi.yaml`



Cosa dire a voce

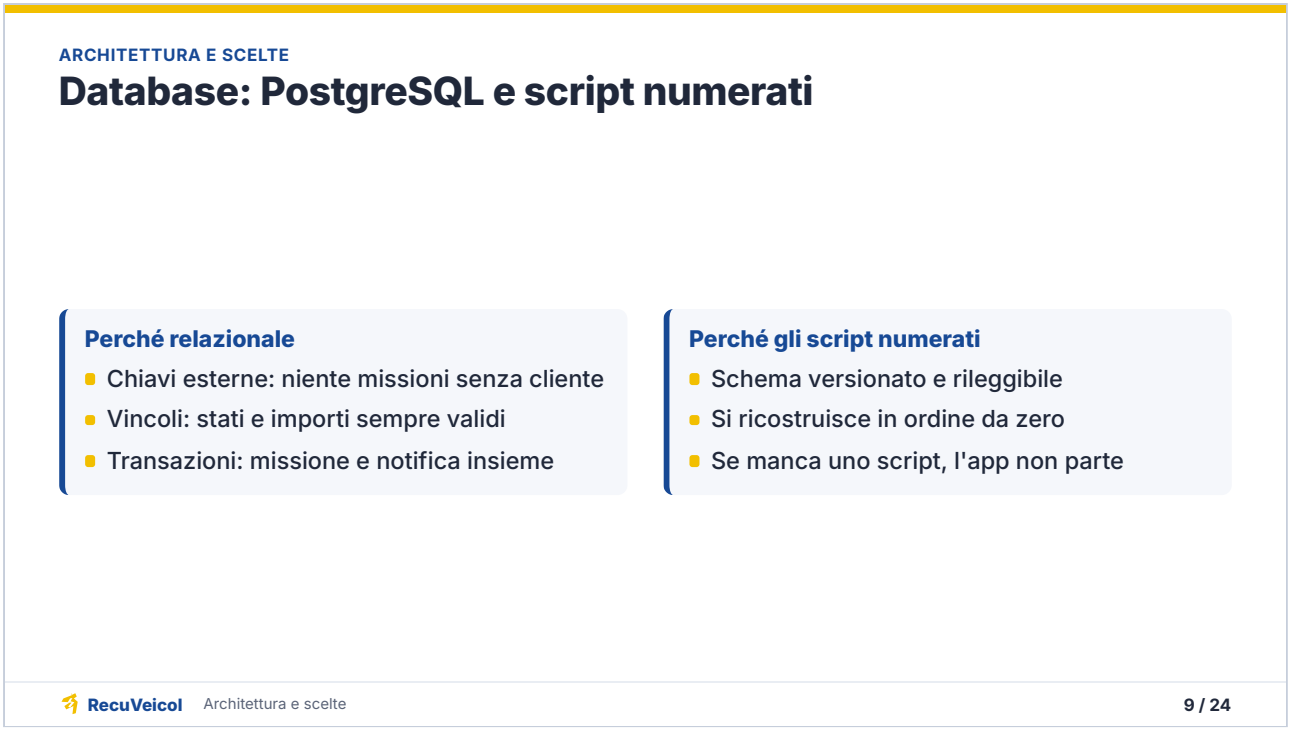
Il backend ha tre livelli. Il controller riceve le richieste, il service applica le regole, il repository parla con il database. Spring Boot ci evita di riscrivere sicurezza, validazione e accesso ai dati. Maven gestisce build e dipendenze, e il wrapper fa usare a tutti la stessa versione.

In parole semplici

Tre strati, ognuno con un solo compito.

File principali

`pom.xml` · `controller/` · `service/` · `repository/`



Cosa dire a voce

I dati sono molto collegati, quindi abbiamo scelto un database relazionale. Le chiavi esterne e i vincoli impediscono dati incoerenti, e le transazioni salvano missione e notifica insieme. PostgreSQL ci permette anche di garantire che un autista abbia una sola missione attiva. Lo schema nasce da script SQL numerati.

In parole semplici

Il database è l'ultima difesa: rifiuta i dati non validi.

File principali

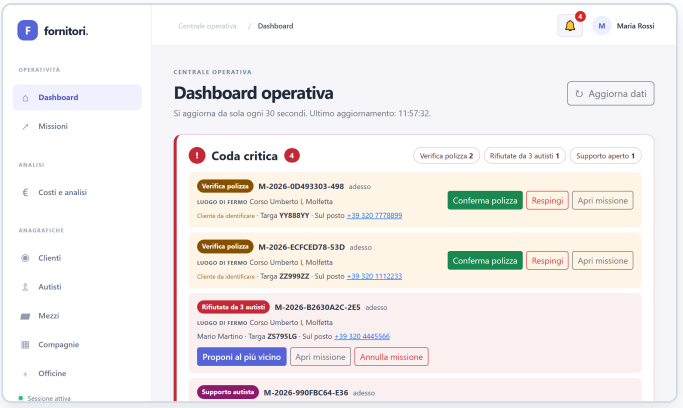
`database/` · `docs/architecture/DATABASE_DESIGN.md`

ARCHITETTURA E SCELTE

Frontend: JavaScript e Leaflet

Perché

- Nessuna compilazione né Node.js
- Pagine servite dal backend
- Leaflet: mappe open source



Dashboard operativa della centrale

Cosa dire a voce

Il frontend usa HTML, CSS e JavaScript, con Bootstrap per il layout e Leaflet per la mappa. Non c'è compilazione: i file li serve direttamente Spring Boot, quindi per avviare tutto basta il backend. Bootstrap rende le pagine adatte anche al telefono, utile per l'autista.

In parole semplici

Il browser riceve file pronti: meno strumenti, avvio più semplice.

File principali

`static/index.html` · `static/app.js` · `static/styles.css`



Cosa dire a voce

Il bot è in Python e usa solo la libreria standard, quindi si avvia senza installare nulla. Abbiamo scelto Telegram perché l'API è gratuita e aperta, non serve l'approvazione di Meta ed è adatta alla demo. Con il pattern outbox la notifica viene salvata insieme alla missione e inviata subito dopo.

In parole semplici

L'outbox è una lista di messaggi da spedire, conservata nel database.

File principali

`chatbot/fornitori_chatbot/engine.py` · `chatbot/fornitori_chatbot/notifications.py`



Cosa dire a voce

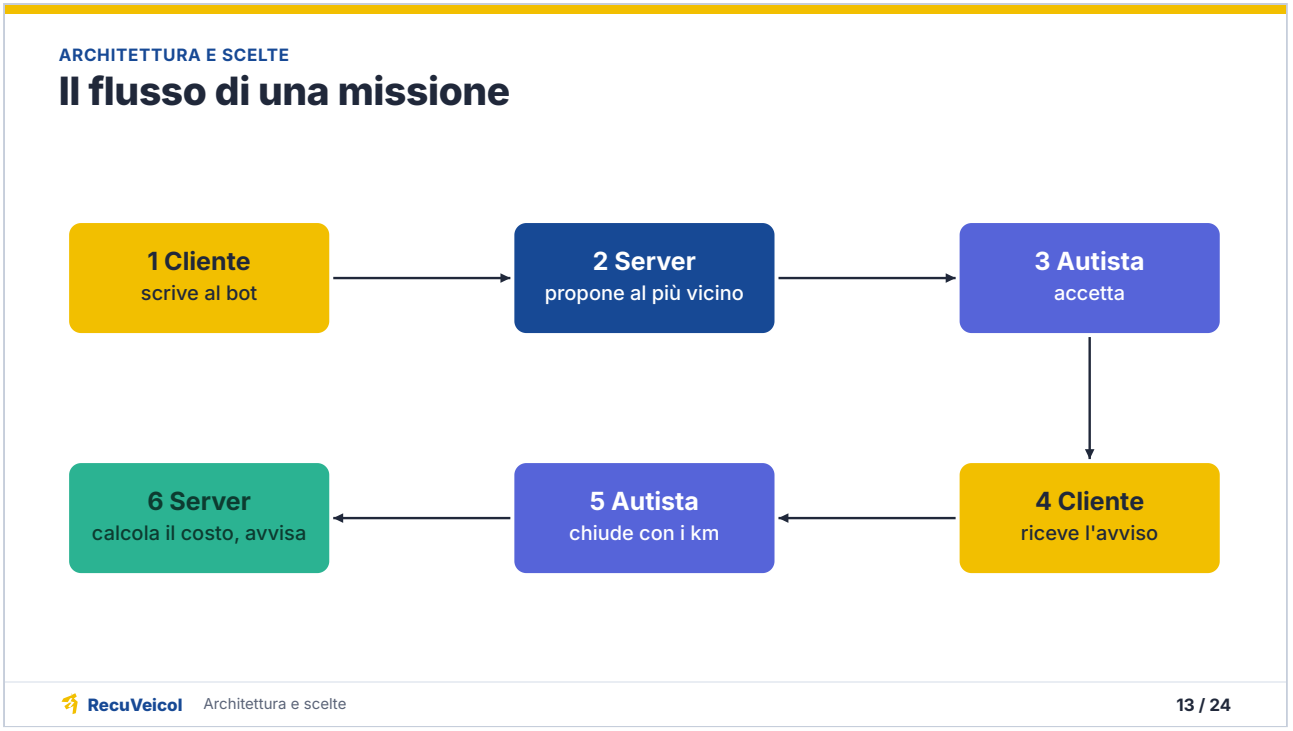
I componenti sono separati e si scambiano dati in JSON tramite API REST, descritte in un unico file OpenAPI. Così ognuno può lavorare sulla propria parte senza rompere le altre. Per le mappe usiamo Nominatim e OSRM, open source e gratuiti, senza chiavi a pagamento.

In parole semplici

Il contratto è l'accordo scritto tra chi offre un'API e chi la usa.

File principali

`api/openapi.yaml` · `docs/team/INTEGRATION_CONTRACTS.md`



Cosa dire a voce

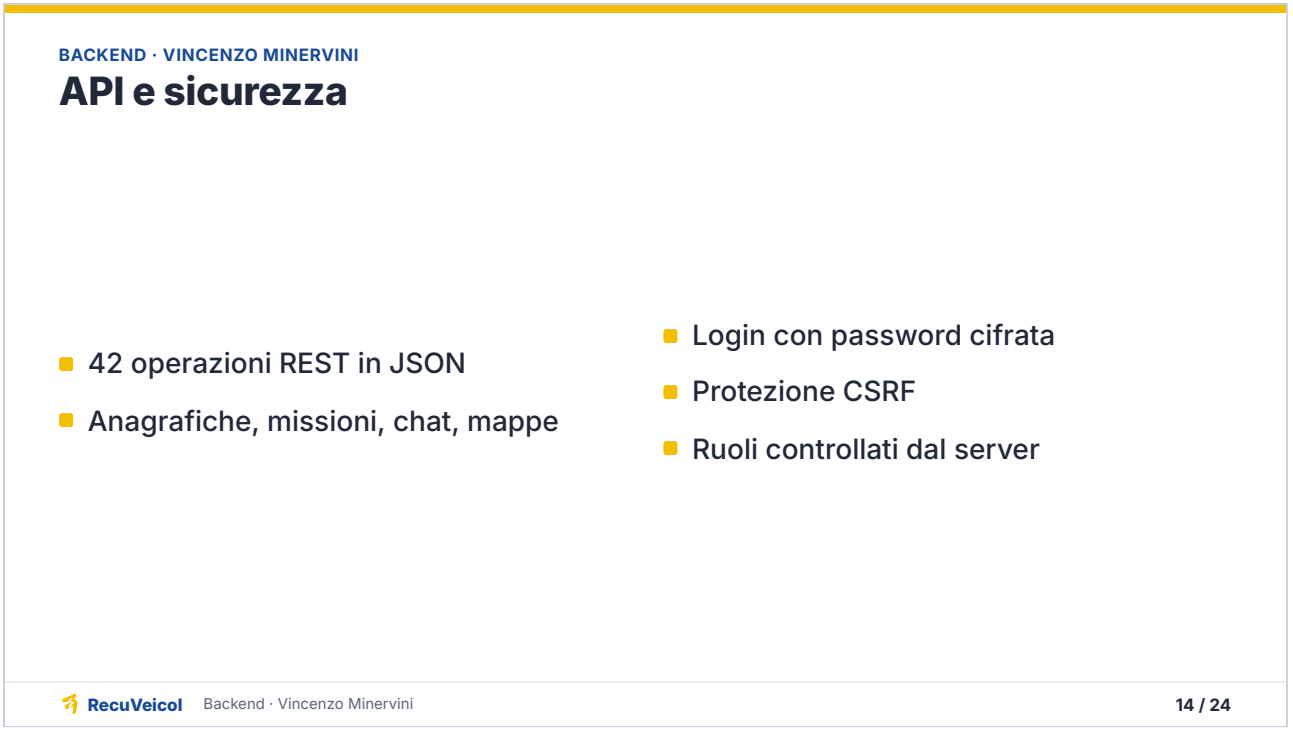
Ecco una missione completa. Il cliente scrive al bot e nasce la richiesta. Il server la propone all'autista più vicino, che ha dieci minuti per rispondere. Quando accetta, il cliente riceve un avviso, senza il suo nome. A fine intervento l'autista inserisce i chilometri, il server calcola il costo e parte l'ultimo avviso.

In parole semplici

Lo stato della missione vive nel backend; bot e pagine lo mostrano.

File principali

`docs/diagrams/mission_states.mmd` · `service/MissionService.java`



Cosa dire a voce

Il backend offre 42 operazioni REST per anagrafiche, missioni, chat e mappe. Per la sicurezza le password sono salvate con BCrypt, un token CSRF impedisce ad altri siti di agire al posto dell'utente e il server controlla il ruolo a ogni richiesta. Il bot ha un token di servizio separato.

In parole semplici

Sessione: chi sei. CSRF: sei stato davvero tu. Ruolo: cosa puoi fare.

File principali

`config/SecurityConfig.java` · `controller/` · `api/openapi.yaml`



Cosa dire a voce

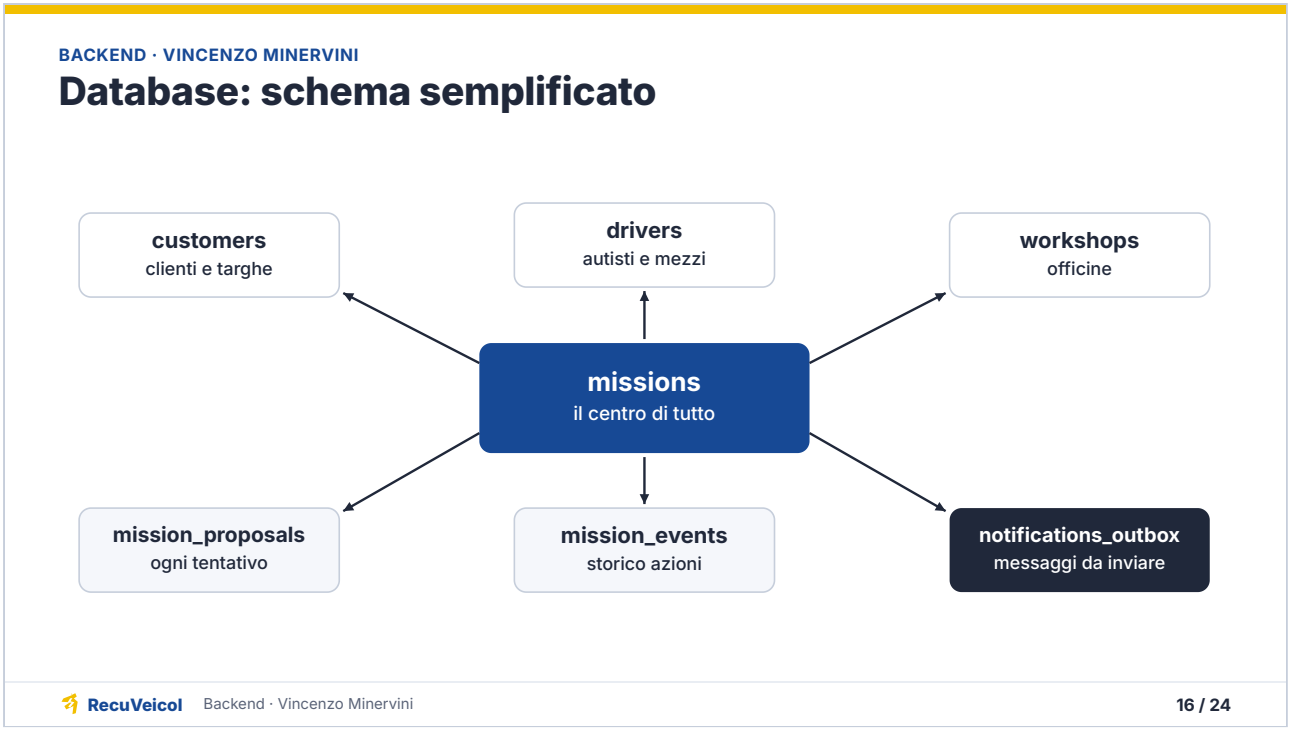
La missione passa da nuova a proposta, accettata ed eseguita; se l'autista rifiuta o non risponde passa al successivo. Dopo tre esiti negativi decide la centrale. La proposta vale dieci minuti. La tariffa è cinquanta euro più un euro e venti al chilometro, con il venticinque per cento in più di notte.

In parole semplici

La fascia 22:00-08:00 è una scelta del team; il PDF del docente indica 20:00-8:00.

File principali

`service/MissionService.java` · `service/PricingService.java` · `service/NearestDriverService.java`



Cosa dire a voce

Al centro c'è la tabella delle missioni, collegata a cliente, autista e officina. Sotto ci sono le proposte, cioè ogni tentativo di assegnazione, lo storico delle azioni e la coda delle notifiche. In tutto le tabelle sono quattordici. Le trenta missioni storiche sono separate da quelle operative.

In parole semplici

La missione è il foglio centrale; le altre tabelle sono schede collegate.

File principali

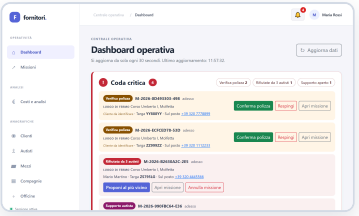
`database/01_schema.sql` · `docs/architecture/DATABASE_DESIGN.md`

FRONTEND · FRANCESCO VALERIANO, GIUSEPPE CANGELLOSI, MARIKA SANCILIO

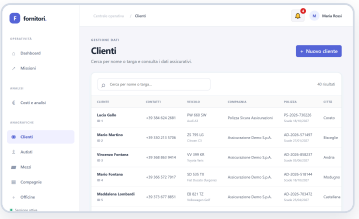
Centrale operativa



Missione rifiutata da 3 autisti: decide la centrale



Dashboard operativa



Anagrafiche

RecuVeicol Frontend · Francesco Valeriano, Giuseppe Cangellosi, Marika Sancilio 17 / 24

Cosa dire a voce

La centrale vede subito la coda critica: polizze da verificare, missioni rifiutate da tre autisti, richieste di supporto. I riquadri aprono l'elenco filtrato. Dalla scheda può proporre, annullare o rettificare il costo, e una campanella segnala rifiuti, scadenze e supporto.

In parole semplici

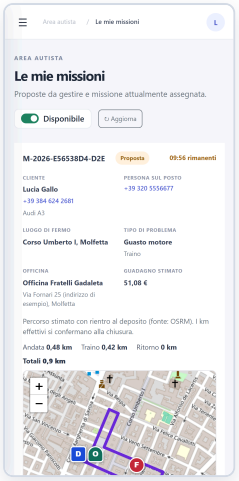
Tutto ciò che è a schermo arriva dal server tramite API.

File principali

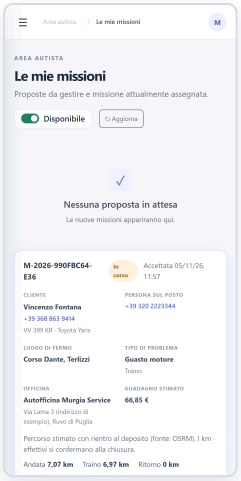
`static/index.html` · `static/app.js` · `controller/MissionController.java`

FRONTEND · FRANCESCO VALERIANO, GIUSEPPE CANGELLOSI, MARIKA SANCILIO

Area autista



Proposta



Dopo aver accettato

- Pensata per il telefono
- Accetta o rifiuta
- Mappa e chilometri

Cosa dire a voce

L'area autista è pensata per il telefono. Le nuove proposte compaiono da sole, con tempo residuo, mappa e guadagno stimato. Nome e telefono del cliente si vedono già dalla proposta, la targa dopo l'accettazione. A fine intervento l'autista conferma i chilometri e chiude la missione, poi trova il totale nello storico.

In parole semplici

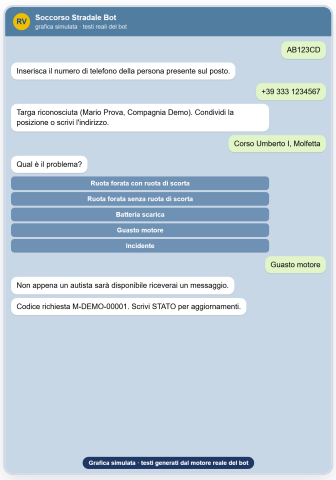
L'autista vede solo le proprie missioni.

File principali

`static/app.js` · `service/MissionService.java`

CHATBOT · MARCO COLUCCI

Il dialogo con il cliente



Testi reali del bot, grafica Telegram simulata

- Targa e telefono sul posto
- Posizione e problema
- Nasce la missione

Cosa dire a voce

Il bot si chiama Soccorso Stradale Bot. Il cliente scrive la targa e il telefono di chi è sul posto. Poi il bot chiede dove si trova e qual è il problema; il tipo di intervento lo decide il backend. Se la polizza non è verificabile interviene la centrale. Alla fine mostra il codice. Scrivendo STATO il cliente può seguire la richiesta.

In parole semplici

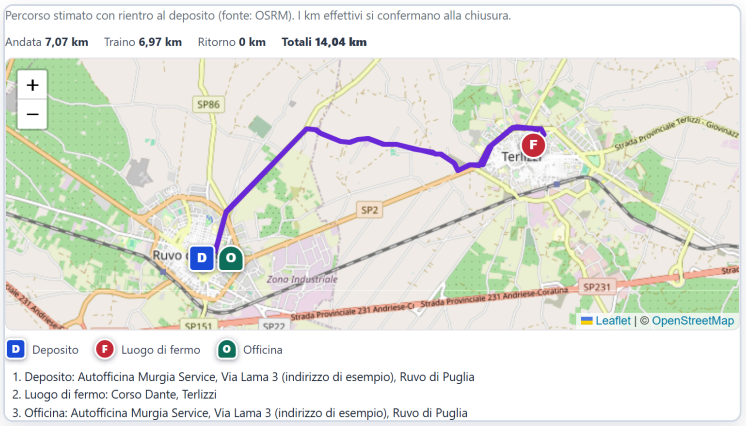
Il bot raccoglie i dati nell'ordine giusto e li passa al backend.

File principali

`chatbot/fornitori_chatbot/engine.py` · `chatbot/fornitori_chatbot/messages.py` · `service/ChatService.java`

MAPPE · MARCO COLUCCI

Mappe: geocoding e percorso



Deposito e officina a Ruvo, luogo di fermo a Terlizzi

- **Nominatim**: da indirizzo a coordinate
- **OSRM**: percorso e km
- **Leaflet**: disegna la mappa

RecuVeicol Mappe · Marco Colucci 20 / 24

Cosa dire a voce

Le mappe usano tre strumenti. Nominatim trasforma l'indirizzo in coordinate. OSRM calcola il percorso su strada e i chilometri stimati. Leaflet disegna tutto nella pagina. In questo traino l'autista parte da Ruvo, raggiunge il veicolo a Terlizzi e lo riporta all'officina di Ruvo.

In parole semplici

I km stimati vengono proposti alla chiusura; il costo usa quelli effettivi.

File principali

`service/MapService.java` · `static/app.js`



Cosa dire a voce

Il cliente riceve un avviso quando un autista prende in carico la richiesta, quando l'intervento è concluso e se viene annullata, mai con il nome dell'autista. Il messaggio viene prima salvato nell'outbox, insieme alla missione. Poi il dispatcher del bot, ogni cinque secondi, prende i messaggi in coda, li invia e registra l'esito.

In parole semplici

Prima si scrive «da inviare», poi il bot spedisce e segna com'è andata.

File principali

`entity/NotificationOutbox.java` · `chatbot/fornitori_chatbot/notifications.py`

CHIUSURA

Scaletta della demo live

#	Dove	Cosa mostriamo
1	Telegram	Nasce la richiesta
2	Centrale	Coda critica e proposta automatica
3	Autista	Proposta, mappa, accetta
4	Telegram	Avviso di presa in carico
5	Autista	Chiusura e costo
6	Centrale	Riepilogo aggiornato

Cosa dire a voce

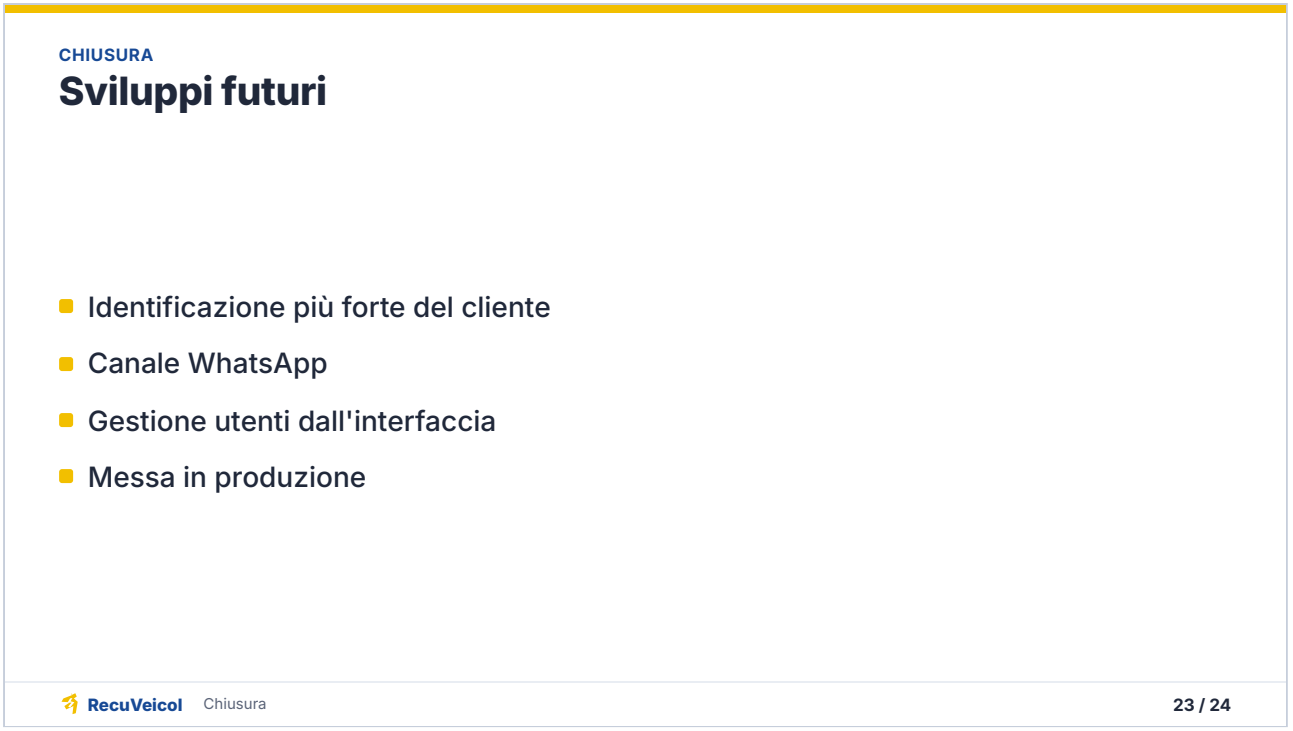
Adesso vediamo il sistema. Su Telegram apriamo una richiesta con una targa dimostrativa. Il sistema la propone all'autista più vicino e la centrale la segue. L'autista, dal telefono, vede la mappa e accetta. Su Telegram arriva l'avviso. Poi chiudiamo con i chilometri e il server calcola il costo.

In parole semplici

Tieni aperte tre finestre: centrale, autista e Telegram.

File principali

`docs/manuals/MANUALE_UTENTE.md` · `chatbot/README.md`



Cosa dire a voce

I prossimi passi sono quattro. Rafforzare l'identificazione del cliente con un secondo dato. Aggiungere WhatsApp, già previsto nel contratto delle API. Gestire gli utenti dall'interfaccia. Infine portare il sistema in produzione, con HTTPS e una configurazione adatta a dati reali.

In parole semplici

Sono tutti sviluppi non ancora implementati, già indicati nei documenti.

File principali

`docs/agile/PRODUCT_BACKLOG.md` · `api/openapi.yaml`



Cosa dire a voce

Abbiamo costruito un sistema che porta una richiesta di soccorso dal primo messaggio fino alla chiusura con il costo calcolato. Regole e costi stanno sul server, e i componenti sono separati e collegati da un unico contratto. Vi ringraziamo per l'attenzione.

In parole semplici

Il messaggio finale in tre punti.

File principali

`README.md` · `docs/00_INDICE_GENERALE.md`